

SkillOpt Reference

Skill format, prompt templates, and implementation hyperparameters

AgentIA Engineering

September 25, 2026

Purpose. This is the implementation companion to the proposal. It reproduces the skill document format, the published prompt templates from SkillOpt (arXiv:2605.23904, Appendix C.2), the CODESKILL skill examples (arXiv:2605.25430, Figures 4–14), and the hyperparameters required to build the loop. Templates are verbatim from the sources so they can be copied directly into the codebase under `backend/app/resources/prompts/skillopt/`.

Contents

1	Optimization in text space	2
1.1	Why it is not obvious	2
1.2	The deep-learning analogy	2
1.3	Why the gate is inside the update	2
1.4	The three failure modes of naive text editing	3
1.5	Why “text space” is the right phrase	3
1.6	How to say it at different depths	3
2	Skill document format	4
2.1	Field semantics	4
2.2	Authoring rules	4
3	Reference skill examples	5
4	SkillOpt prompt templates	6
4.1	Failure analysis — <code>analyst_error.md</code>	6
4.2	Success analysis — <code>analyst_success.md</code>	6
4.3	Failure merge — <code>merge_failure.md</code>	7
4.4	Success merge — <code>merge_success.md</code>	7
4.5	Cross-priority merge — <code>merge_priority.md</code>	7
4.6	Ranking and selection — <code>ranking.md</code>	7
4.7	Epoch-wise slow update — <code>slow_update.md</code>	8
4.8	Optimizer meta skill — <code>meta_skill.md</code>	8
5	CODESKILL supplementary templates	8
5.1	Task-level extraction — Figure 6	9
5.2	Event-driven extraction — Figure 7	9
5.3	Evolution — Figure 8	9
5.4	Bank maintenance — Figure 9	10
6	Hyperparameters	10
6.1	AgentIA adaptations	10
7	Implementation checklist	10
8	Sources	11

1 Optimization in text space

The one-sentence version. Take the four things that make weight training reliable — a bounded step size, a validation gate, negative feedback, and momentum — and apply them to a text document instead of to a matrix of numbers. Everything else in this reference is a detail underneath that sentence.

1.1 Why it is not obvious

Before SkillOpt, there were two ways to improve an LLM’s guidance text.

Hand-editing. A human writes a prompt, tests it, edits it, tests again. Works, but does not scale, and the human’s judgment is the only signal.

Self-editing. The model reads its own failures and rewrites its own prompt. Scales, but is unreliable — no bound on how much it rewrites, no validation that the rewrite helped, no memory of what was tried and failed. In practice, self-editing drifts: the document grows longer, more contradictory, and eventually worse.

SkillOpt is a third option: self-editing *with the discipline of an optimizer*. Bounded edits, validated against held-out data, with negative feedback and momentum. It borrows every control that makes gradient descent reproducible, and applies those controls to text.

1.2 The deep-learning analogy

The analogy is operational, not decorative. Every row matters.

Deep learning	SkillOpt	What it prevents
Weights	The skill document	(the trainable object)
Forward pass	Rollout batch on training tasks	Editing from too little evidence
Loss	Task score (0/1 from verifier)	(the objective)
Gradient	Reflection — LLM proposes edits	(the proposed direction)
Learning rate	Edit budget L_t per step	Unbounded rewriting
Optimizer step	Apply bounded edits $\rightarrow$ candidate	(the move)
Validation set	Held-out selection split	Overfitting to motivating examples
Acceptance rule	Strict improvement, ties rejected	Silent drift and degradation
Momentum	Epoch-wise slow/meta update	Long-horizon drift
Rejected gradients	Rejected-edit buffer	Repeated bad proposals

The analogy is the argument: if you believe in gradient descent, you should believe in bounded text editing with a validation gate, because they are the same algorithm in different spaces.

1.3 Why the gate is inside the update

This is the part a technical judge will care about. In standard SGD:

`weights  $\leftarrow$  weights - lr  $\cdot$  gradient` (always take the step)

You always move. Validation runs separately to pick checkpoints.

In SkillOpt:

```
candidate  $\leftarrow$  apply_bounded_edits(skill)
if score(candidate) > score(current):
    skill  $\leftarrow$  candidate
```

```

else:
    reject, log to buffer

```

The step is conditional on validation. The gradient (the LLM’s reflection) proposes; the held-out score disposes. This is closer to a *trust-region* or *line-search* method than to vanilla SGD — and it is necessary because the “gradient” here is a language model’s guess, not a computed direction. A language model’s reflection can look plausible and be wrong. The gate is what makes an unreliable proposer safe.

The consequence: the loop is monotone. The skill’s selection score never goes down, by construction. Every accepted edit is a measurable improvement on held-out data.

1.4 The three failure modes of naive text editing

If someone pushes back with “isn’t this just prompt engineering,” answer with the three ways naive editing fails — each corresponding to a control SkillOpt adds.

1. **Unbounded editing.** Naive self-editing rewrites the whole document. You lose the ability to attribute improvement to any specific change, and the document drifts. SkillOpt bounds each step to L_t edits (default 4), preserving continuity between versions.
2. **Unvalidated editing.** Naive self-editing accepts any plausible rewrite. A plausible textual diagnosis can still hurt the actual model. SkillOpt requires strict improvement on a held-out split before any candidate becomes current.
3. **No momentum.** Naive self-editing only compares adjacent versions. It cannot see slow drift across many steps. SkillOpt re-runs the *same tasks* across epoch boundaries and writes durable guidance to a protected section that step-level edits cannot touch.

The paper’s ablation is the empirical proof: removing the momentum mechanisms (meta skill + slow update) drops SpreadsheetBench from 77.5 to 55.0 — a 22.5-point degradation. That is the cost of skipping the control.

1.5 Why “text space” is the right phrase

The optimization is literally happening over the space of possible skill documents. Each document is a point in that space. Each edit is a move. The validation score is the objective. The gate keeps you on the improving side.

The subtlety: this space is **non-differentiable and open-ended**. You cannot take a gradient, and the set of possible edits is anything a language model can write. SkillOpt replaces the gradient with a language-model reflection, and replaces the step-size schedule with an edit-count budget. Everything else — validation, momentum, negative feedback — is directly imported.

1.6 How to say it at different depths

Ten seconds. “It’s a training loop for a text document. Each round, the model proposes small edits, and the edit is only kept if it measurably improves performance on a held-out set. Same discipline as training a neural network — just applied to a prompt instead of to weights.”

Thirty seconds. “You have a skill document that gets prepended to the generation prompt. An optimizer model reads rollout trajectories and proposes atomic edits — append, insert, replace, delete — capped at a fixed number per step. Each candidate document is scored on a held-out split, and accepted only if the score strictly improves. Rejected edits go into a buffer so the next round does not repeat them. At each epoch, the same tasks are re-run under the old and new documents, and durable guidance gets written to a protected section. The controls — learning rate, validation, momentum, negative feedback — are exactly the ones you would want in SGD.”

Two minutes. The four-control mapping above, plus the trust-region framing of the gate, plus the 22.5-point ablation as evidence that the controls matter.

The three sentences to have ready. (1) *The skill document is the trainable parameter, and the model stays frozen.* No weights touched, no fine-tuning, no training infrastructure — just text. (2) *The step is validated before it is taken.* Every candidate must strictly beat the current version on a held-out set. This is what makes self-editing safe where naive self-editing drifts. (3) *The four controls — bounded edits, validation gate, rejected-edit buffer, epoch-wise momentum — are exactly the controls that make weight-space optimization reproducible, applied to text space.*

2 Skill document format

Every skill is a standalone markdown file. This format is the convergence point of all three 2026 papers (SkillOpt, CODESKILL, SkillBrew); it is the contract between authoring, injection, and optimization.

```
# Title
# <short reusable name -- no repo, class, file, or variable names>

## Granularity
task-level | event-driven

## When to apply
<transferable trigger condition -- high-level, not instance-specific>

## Rules
1. <actionable rule>
2. <actionable rule>
...

<!-- SLOW_UPDATE_START ->
<!-- SLOW_UPDATE_END ->
```

2.1 Field semantics

- **Title.** Short, reusable, capability-oriented. Describes *what the skill enables*, not a specific instance of it.
- **Granularity.** Two values. *Task-level* for whole-task strategies (e.g., “diagnose Maven build failures”). *Event-driven* for local trigger-response patterns (e.g., “handle missing test-jar dependency”).
- **When to apply.** The trigger condition. High-level enough to fire on unseen tasks of the same family; specific enough not to fire on every task. This is the hardest field to author well.
- **Rules.** Ordered, procedural, actionable. Each rule should describe a concrete action or check. Prefer numbered steps over prose.
- **Protected section.** Empty in seed skills. Only the epoch-wise slow/meta update (see §4.7) may write here. Step-level edits are blocked from touching it by design.

2.2 Authoring rules

1. Rules must be **procedural**, not declarative.
2. Rules must be **ordered** when sequencing matters.
3. Trigger conditions must be **transferable**.

4. **No instance-specific identifiers.** No repo names, file paths, class names, module names, or one-off literals.
5. Every rule must be **grounded** in observed failure evidence or constitutional principles.
6. Combined length per skill: **under 500 tokens**.
7. Combined length of all seed skills: **under 2,000 tokens**.

3 Reference skill examples

Two examples from CODESKILL's maintained bank. They are Java/Maven specific and represent the target quality bar for AgentIA's seed skills.

Task-level example

```
# Diagnose Maven build failures via environment, config, and dependency resolution.
## Granularity
task-level
## When to apply
When a Java/Maven project fails to build and the cause may involve toolchain or
wrapper configuration, repository settings, missing or stale dependencies, or
build-phase ordering rather than source-code errors.
## Rules
1. Identify the project type and build instructions from repository metadata before
making changes.
2. Run the build with the project's Maven wrapper first, then capture both early and
later output to distinguish dependency-download progress from the actual failure.
3. Use the first concrete failure message to distinguish network or repository
issues, missing artifacts, and compilation errors before changing code.
4. If the build fails in the enforcer or toolchain phase, use verbose output to
identify the required Java and Maven versions.
5. Check active Java and Maven toolchain versions and align them with the project's
documented requirements.
6. When a dependency cannot be resolved, check whether the missing artifact is
generated later in the build.
7. Inspect local Maven settings and repository configuration before changing code.
8. After adjusting toolchain, wrapper, repository, or dependency configuration,
rerun and re-check the logs to confirm the failure mode changed.
```

Event-driven example

```
# Handle missing Maven test-jar dependencies.
## Granularity
event-driven
## When to apply
When a Maven build fails with a dependency resolution error for a test-jar or similar
scoped artifact that has not yet been produced or installed.
## Rules
1. Inspect the failing module's POM to confirm dependency type and scope.
2. Check the producing module's POM for an existing execution for the required
artifact.
3. If the test jar is only generated when tests run, either run the tests or
explicitly invoke the Maven test-jar goal before rebuilding.
4. If the artifact is missing, add or configure the appropriate plugin execution in
```

```

the producing module's POM.
5. Do not retry the same Maven install command with tests skipped; skipping prevents
the test jar from being created.
6. After generating or configuring the artifact, run a full install so it is
installed into the local repository.
7. Verify the artifact is present in the local Maven repository before retrying the
original command.

```

4 SkillOpt prompt templates

All templates below are reproduced verbatim from SkillOpt, Appendix C.2, with terminology aligned to the optimizer/target framing. They are designed to be dropped into `prompts/skillopt/` as-is. Prompt outputs are JSON so the pipeline can parse, filter, apply, and validate without manual intervention.

4.1 Failure analysis — `analyst_error.md`

```

You are an expert failure-analysis agent for AI agent tasks.
You will be given MULTIPLE failed agent trajectories from a single minibatch and the
current skill document. Your job is to identify the most important COMMON failure
patterns across the batch and propose a concise set of skill edits.

## Analysis Process
1. Read ALL trajectories in the minibatch.
2. Identify the most prevalent, systematic failure patterns.
3. For each pattern, classify its failure type.
4. Propose skill edits that address the COMMON patterns, not individual edge cases.
5. Edits must be generalizable; do not hardcode task-specific values.
6. Only patch gaps in the skill; do not duplicate existing content.

You will be told the maximum number of edits (the budget L). Produce AT MOST L edits.
Respond ONLY with a valid JSON object:
{
  "reasoning": "<why these edits address the batch's common failures>",
  "edits": [
    {"op": "append", "content": "<markdown to add at end of skill>"},
    {"op": "insert_after", "target": "<exact heading/text>", "content":
"<markdown>"},
    {"op": "replace", "target": "<exact text>", "content": "<replacement>"},
    {"op": "delete", "target": "<exact text>"}
  ]
}

```

4.2 Success analysis — `analyst_success.md`

```

You are an expert success-pattern analyst for AI agents.
You will be given MULTIPLE successful agent trajectories from a single minibatch and
the current skill document. Identify generalizable behavior patterns COMMON across
the batch worth encoding in the skill.

## Rules
- Only propose patches for patterns NOT already covered.
- Focus on patterns appearing across MULTIPLE trajectories.
- Patterns must generalize beyond specific tasks.
- Prefer reinforcing existing sections over adding new ones.

Produce AT MOST L edits. Output: same JSON schema as failure analysis with a

```

"success_patterns" array added.

4.3 Failure merge — merge_failure.md

You are a skill-edit coordinator. Merge multiple independently-proposed patches from FAILURE analysis into ONE coherent patch.

Merge guidelines:

1. Deduplicate: keep the best-worded version of similar edits.
2. Resolve conflicts: choose the one with stronger justification or synthesize both.
3. Preserve unique insights: include all non-redundant corrective edits.
4. Prevalent-pattern bias: edits appearing across multiple patches get HIGH priority.
5. Independence: no two edits may target the same text region.
6. Support count: for each merged edit, estimate how many source patches support it.
7. PROTECTED SECTION: do not target content between SLOW_UPDATE_START and SLOW_UPDATE_END.

4.4 Success merge — merge_success.md

Same structure as failure merge, applied to success-driven patches. Be conservative: success patches reinforce existing behavior; only include edits for patterns not already in the skill.

4.5 Cross-priority merge — merge_priority.md

Merge failure and success patches.

1. FAILURE PATCHES TAKE PRIORITY: failure-driven edits are preserved unless they directly conflict with a well-supported success pattern.
2. Deduplicate: when a failure and success edit cover the same point, keep the failure version.
3. Preserve success insights for patterns not addressed by failures.
4. Higher-level merges represent broader consensus; give priority.
5. Carry forward support_count and source_type for each edit.
6. PROTECTED SECTION: do not target protected content.

4.6 Ranking and selection — ranking.md

You are an expert edit-ranking optimizer for a skill optimization system. Rank the edits and select the top ones.

Ranking criteria (in priority order):

1. Systematic impact: edits addressing widespread, recurring failure patterns rank highest. A rule fixing 50% of failures beats one fixing a single edge case.
2. Complementarity: edits filling gaps, not duplicating content.
3. Generality: general principles rank higher than instance-specific advice.
4. Actionability: concrete guidance ranks higher than vague advice.

Respond ONLY with:

```
{"reasoning": "...", "selected_indices": [<0-based indices>]}
```

4.7 Epoch-wise slow update — slow_update.md

You are a strategic skill advisor for an AI agent optimization system. You see how the skill has evolved across an entire epoch by comparing the SAME tasks under two consecutive skill versions. This longitudinal view lets you identify systemic drift, regressions, and persistent blind spots that step-level edits cannot catch.

What you receive

1. Previous epoch's skill and current epoch's skill.
2. Longitudinal comparison: same 20 training tasks under both skills, categorized into regressions, persistent failures, improvements, and stable successes.
3. Previous slow update guidance, if any.

Your process

1. Reflect on the previous guidance: what was effective, what backfired, what was missed.
2. Write updated guidance that retains what worked, revises what didn't, adds new instructions for observed regressions and persistent failures.

Output requirements

Write a strategic guidance block that will **OVERWRITE** the previous guidance in the protected section. Prioritize: (1) preventing regressions, (2) fixing persistent failures, (3) reinforcing successful patterns. Do NOT duplicate content already in the main skill body.

Respond **ONLY** with:

```
{"reasoning": "...", "slow_update_content": "<exact text>"}
```

4.8 Optimizer meta skill — meta_skill.md

You are an optimizer coach. Your job is to write a compact optimizer-side meta skill that helps future optimizer calls produce better skill edits in this environment.

What you receive

1. Previous epoch's last-step skill.
2. Current epoch's last-step skill.
3. Longitudinal comparison on the SAME sampled tasks.
4. Previous optimizer memory, if one existed.

Your goal

Capture: which kinds of edits help in this environment; which tend to be vague, redundant, brittle, or harmful; what level of abstraction works best; which failure-repair patterns to prioritize; which regression risks to guard against.

Constraints

- Address the FUTURE OPTIMIZER directly, not the training model.
- Use evidence from the adjacent-epoch comparison, not generic advice.
- Keep it compact and high-signal.

Respond **ONLY** with:

```
{"reasoning": "...", "meta_skill_content": "<compact guidance>"}
```

5 CODESKILL supplementary templates

CODESKILL's prompt set (Figures 6–14) is complementary to SkillOpt's. Use CODESKILL's extraction and maintenance prompts for the initial skill authoring pass, and SkillOpt's prompts for the optimization loop.

5.1 Task-level extraction — Figure 6

1. A bootstrap skill is a task-level reusable pattern supported by multiple trajectories. It must be broader than a single local event.
2. Generate only if the pattern is reusable across similar tasks. Keep when_to_apply high-level and write transferable, actionable rules.
3. Ground every part of the skill in repeated evidence from the trajectories and outcomes. Do not invent unsupported steps, checks, or guidance. If failed trajectories reveal reusable cautions, include them as cautionary rules.
4. Skip if the evidence is weak, contradictory, accidental, too local, or collapses into an event-level reaction instead of a task-level pattern.
5. Do not include repository names, issue descriptions, exact task goals, variable names, function names, class names, module names, exact file paths, or one-off literals.

Output schema:

```
{
  "action": "generate",
  "skill": {
    "title": "...",
    "granularity": "general",
    "when_to_apply": "...",
    "rules": [...]
  }
}
or {
  "action": "skip",
  "reason": "..."
}
```

5.2 Event-driven extraction — Figure 7

You are an expert at extracting reusable memory from bash-agent trajectories. Extract one reusable event-driven skill from a full trajectory. Use only the provided evidence.

Rules

1. An event-driven skill is a local trigger-response pattern. Focus on one important event; stay narrower than a whole-task workflow.
2. Generate only if the event yields a reusable lesson beyond this exact task.
3. Ground the trigger and guidance in the trajectory and result. Do not invent an event.
4. If multiple candidate events exist, choose the single most reusable one.
5. Skip if there is no strong reusable local event, or if the lesson is too task-specific or workflow-level.
6. No repository names, file paths, or one-off literals.

Output: same schema with "granularity": "event-driven".

5.3 Evolution — Figure 8

Choose exactly one action: evolve (revise exactly one existing skill) or skip (make no revision).

Rules

1. Alignment and need: choose evolve only when one existing skill is clearly aligned with the current trajectory and the result shows it should be revised.
2. Reusability: revise only if the trajectory reveals a reusable missing case, missing check, better decision rule, improved ordering, or reusable caution.
3. Applicability: when_to_apply remains high-transferable.
4. Grounding: the revision must be directly supported by the provided trajectory and result.
5. Identity preservation: keep the same capability identity.
6. Skip if the evidence is weak, contradictory, or the trajectory is better described as a brand-new skill.

5.4 Bank maintenance — Figure 9

Choose exactly one action: add, merge, or drop.

1. add: candidate is reusable, coherent, and distinct from retrieved skills.
2. merge: candidate and one retrieved skill share the same capability identity and can be combined into one stronger skill.
3. drop: candidate is covered by stronger skills, redundant, weakly evidenced, unsafe, too local, or too task-specific.
4. No resulting skill may include repo names, file paths, or one-off literals.

6 Hyperparameters

Default configuration for the SkillOpt loop. Values from Table 4 of the paper, with AgentIA-specific adaptations noted.

Parameter	Value	Notes
Edit budget L_t	4	cosine decay, floor 2
Rollout batch size	40	scale to session volume
Reflection minibatch	8	fixed
Epochs	4	fixed
Slow-update samples/epoch	20	fixed
Optimizer model	frontier	different provider than target
Gate criterion	strict >	ties rejected

6.1 AgentIA adaptations

- **Verifier.** `mvn test -o exit code` + constitutional validators. The paper assumes a scalar verifier; the composite score is AgentIA-specific.
- **Selection split.** 20% of recent sessions, never used for training or authoring. Minimum 10 sessions; prefer 15.
- **Reflection batch.** Scale the paper’s 40 down to session volume (30 is realistic for the pilot).
- **Optimizer model.** A different provider via `LLMFactory` to avoid correlated generate-and-critique failure.

7 Implementation checklist

1. Copy all prompt templates from §4 into `backend/app/resources/prompts/skillopt/` as separate `.md` files.
2. Author seed skills using the format in §2 and the examples in §3. Save to `backend/app/resources/skills/`.
3. Implement the edit applier with the four atomic operations (append, insert_after, replace, delete). Enforce the protected section guard.
4. Implement the hierarchical merge in the order: failure merge, success merge, cross-priority merge.
5. Implement the ranker that clips to L_t .
6. Implement the gate: score the candidate on the selection split, accept only if strictly greater than current.
7. Persist rejected edits with score drops to the buffer.

8. Implement the epoch-boundary slow update; write to the protected section.
9. Implement the meta-skill capture; keep optimizer-side only.
10. Wire the loop: forward pass on the training split, reflection minibatches, merge, rank, apply, gate, repeat.

The one thing not to skip. The epoch-wise slow/meta update (§4.7). SkillOpt’s ablation shows removing both the meta skill and the slow update costs 22.5 points on SpreadsheetBench — the largest single degradation in the paper. Per-step reflection cannot substitute for it.

8 Sources

- Li et al. *SkillOpt: A controllable text-space optimizer for agent skills*. arXiv:2605.23904 (2026). Appendix C.2 is the source for all prompts in §4; Table 4 for hyperparameters.
- Li et al. *CODESKILL: Learning self-evolving skills for coding agents*. arXiv:2605.25430 (2026). Figures 4–14 for skill format, examples, and extraction/evolution/maintenance prompts.
- Hu et al. *SkillBrew: Multi-objective curation of skill banks*. arXiv:2605.29440 (2026). Referenced for deferred bank-level curation.